

SIMFAT – Sistema Integrado de Monitoreo y Alerta Temprana Forestal

Informe Consolidado – Estado de Avance N°3

Evaluación Parcial N°3 TPY1101 – Taller Aplicado de Programación Escuela de
Informática y Telecomunicaciones Duoc UC

Integrantes:

- Andrés Ibáñez
- David Vásquez

Cliente: AIFBN – Agrupación de Ingenieros Forestales por el Bosque
Nativo **Fecha:** Junio 2026 **Versión:** 3.0

Índice

1. Introducción
2. Resumen Ejecutivo EP1 y EP2
 - 2.1 EP1 – Propuesta Inicial (semanas 1-4)
 - 2.2 EP2 – Estado de Avance N°2 (semana 10)
3. Desarrollo EP3
 - 3.1 Metodología de desarrollo aplicada (EP3)
 - 3.2 Estado del producto al inicio de EP3
 - 3.3 Arquitectura del sistema (al cierre de EP3)
 - 3.4 Módulos implementados en EP3
 - 3.4.1 Módulo Territorial
 - 3.4.2 Panel Analítico Regional (Dashboard)
 - 3.4.3 Módulo de Alertas y Reglas
 - 3.4.4 Módulo Comunitario
 - 3.4.5 Panel de Accesos RBAC
4. Plan de Pruebas de Software (EP3)
 - 4.1 Descripción del ambiente de pruebas
5. Pruebas de Validación Aplicadas
 - 5.1 Resultados de validación por componente
 - 5.2 Resumen de resultados
6. Mejoras Implementadas EP3
7. Documentación Técnica
 - 7.1 Diagrama de clases (EP3)
 - 7.2 Modelo de base de datos (MER)
 - 7.2.1 PostgreSQL – Identidad y acceso (RBAC)
 - 7.2.2 MongoDB – Colecciones de negocio (nuevas en EP3)
 - 7.3 Guía de instalación y despliegue
 - 7.4 Control de versiones

8. Estado MVP en Producción
9. Costos y Sostenibilidad
10. Conclusiones
11. Lecciones Aprendidas
12. Anexos

1. Introducción

SIMFAT (Sistema Integrado de Monitoreo y Alerta Temprana Forestal) es una plataforma web desarrollada como proyecto de titulación de la asignatura TPY1101 – Taller Aplicado de Programación, en la Escuela de Informática y Telecomunicaciones de Duoc UC. El sistema fue diseñado a solicitud de la AIFBN (Agrupación de Ingenieros Forestales por el Bosque Nativo), organización que enfrenta el desafío de monitorear el riesgo de incendio forestal en Chile a través de múltiples regiones y comunas, con datos provenientes de distintas fuentes institucionales y satelitales.

El problema central que SIMFAT aborda es la dispersión de los datos forestales y meteorológicos disponibles para los coordinadores territoriales. Antes de SIMFAT, un coordinador que quisiera evaluar el riesgo de incendio en una región determinada debía consultar manualmente el portal de NASA FIRMS, las plataformas de datos meteorológicos de Chile, y los informes Copernicus de manera independiente, sin una visión integrada ni un mecanismo de alerta temprana automática.

SIMFAT integra estas fuentes en una única plataforma con mapa interactivo, panel analítico, sistema de alertas configurables y módulo de coordinación comunitaria, permitiendo a los coordinadores forestales tomar decisiones preventivas informadas con datos actualizados cada 12 horas y confirmación satelital on-demand.

El presente informe corresponde al Estado de Avance N°3 (EP3), que cubre el período de la semana 10 a la semana 15 del semestre. En EP3 se completaron los módulos funcionales pendientes, se integró el análisis territorial y comunal con datos satelitales en tiempo casi real, se implementó un ciclo formal de pruebas con 34 casos de prueba, y se ejecutaron 9 mejoras directamente derivadas del proceso de QA.

Este documento funciona como informe consolidado del estado del proyecto al cierre de EP3. Un lector sin acceso al repositorio debe ser capaz de comprender la

arquitectura del sistema, los módulos implementados, el estado de calidad del producto y los pasos para reproducir el ambiente de desarrollo.

2. Resumen Ejecutivo EP1 y EP2

2.1 EP1 – Propuesta Inicial (semanas 1-4)

En EP1 el equipo identificó el problema, definió la solución y planificó el proyecto completo.

Problema identificado: Los datos forestales disponibles para los coordinadores de AIFBN estaban dispersos entre múltiples plataformas sin integración: NASA FIRMS para focos activos, portales meteorológicos para FWI, Copernicus para NDVI y NDMI, y sistemas locales para reportes ciudadanos. La toma de decisiones era reactiva, basada en informes manuales con latencia de días o semanas.

Solución propuesta: Una plataforma web con las siguientes capacidades:

- Integración de múltiples fuentes de datos forestales y meteorológicos en un único panel
- Análisis predictivo mediante score de riesgo compuesto (WLC) por región y comuna
- Sistema de alertas tempranas configurables con umbrales por indicador
- Módulo de coordinación comunitaria para coordinadores y ciudadanía organizada
- Acceso diferenciado por roles (RBAC) para distintos tipos de usuarios

Stack tecnológico seleccionado:

- Frontend: React 18 + Vite
- Backend: Java 21 + Spring Boot 3
- Base de datos relacional: PostgreSQL (identidad y acceso)
- Base de datos documental: MongoDB Atlas (datos de negocio)
- Servicio analítico satelital: Python + FastAPI (integración OpenEO/Copernicus)

Análisis de mercado: Se evaluaron tres alternativas existentes: Global Forest Watch (GFW) de World Resources Institute, el portal NASA FIRMS, y soluciones

SIG institucionales. Ninguna de estas alternativas ofrecía integración local adaptada al contexto forestal chileno, ni mecanismos de alerta temprana configurables para coordinadores regionales, lo que justificó el desarrollo de SIMFAT como solución específica.

Planificación: El proyecto se dividió en tres fases: Inicio (EP1, semanas 1-4), Desarrollo (EP2 y EP3, semanas 5-15) y Cierre (semana 16+). Cada fase incluyó entregas parciales con documentación formal y ciclos de QA.

2.2 EP2 – Estado de Avance N°2 (semana 10)

En EP2 el equipo entregó los cimientos de seguridad del sistema y el MVP inicial desplegado en producción.

Logros principales de EP2:

- **Implementación RBAC data-driven:** Los roles y permisos del sistema se almacenan en PostgreSQL como datos, no como constantes en el código. Esto permite agregar o modificar permisos sin necesidad de redeploy del backend. Se definieron 5 roles: ROLE_SUPER_ADMIN, ROLE_ADMIN, ROLE_MODERATOR, ROLE_VERIFIED_USER y ROLE_COMMUNITY_USER.
- **JWT stateless con AuthorizationResolverService:** Autenticación basada en JSON Web Tokens sin estado de sesión en el servidor. El servicio AuthorizationResolverService resuelve los permisos efectivos del usuario a partir de sus roles en PostgreSQL y los incluye en el token.
- **Panel de control de accesos frontend operativo:** Interfaz para que administradores gestionen usuarios, asignen roles y revisen el historial de verificación de identidad.
- **Integración OpenEO/Copernicus:** Servicio Python independiente (openeo-service) que consulta la plataforma Copernicus Data Space Ecosystem para obtener índices NDVI y NDMI satelitales por región. Desplegado en Railway.
- **MVP online en producción:** Frontend desplegado en Vercel, backend y openeo-service en Railway, con MongoDB Atlas y PostgreSQL Supabase como bases de datos en la nube.
- **Swagger/OpenAPI habilitado:** Documentación interactiva de la API REST disponible en producción para validación de contratos.

- **Pruebas de integración:** SecurityAuthorizationIntegrationTest para validar el sistema RBAC y OpenApiSwaggerIntegrationTest para verificar la consistencia del contrato de la API.

3. Desarrollo EP3

3.1 Metodología de desarrollo aplicada (EP3)

SIMFAT utiliza un modelo de **Prototipado Evolutivo** en el que cada estado del avance entrega un producto funcional que evoluciona progresivamente a partir de la retroalimentación del cliente y de los ciclos de QA internos.

En EP3, el foco metodológico fue distinto al de EP2: mientras EP2 se concentró en los cimientos de seguridad y autenticación, EP3 se orientó a completar los módulos funcionales de cara al usuario final. Las actividades centrales del período fueron:

1. **Completar los módulos funcionales pendientes:** módulo territorial con análisis WLC por comuna, panel analítico regional, módulo de alertas con reglas configurables, módulo comunitario con tablero, biblioteca y agenda.
2. **Integrar fuentes de datos satelitales en el mapa interactivo:** capas FIRMS, NDVI, NDMI y viento con datos de Open-Meteo en el mapa Leaflet.
3. **Aplicar un ciclo formal de pruebas:** definición de 34 casos de prueba organizados por caso de uso, categoría de seguridad y usabilidad, con ejecución documentada.
4. **Ejecutar mejoras derivadas del QA:** 9 mejoras trazadas a casos de prueba específicos, cubriendo corrección de bugs, completitud funcional y usabilidad.

El equipo operó con un ciclo semanal de revisión: David Vásquez como desarrollador principal y Andrés Ibáñez como responsable de QA, revisión de interfaz y validación de criterios de aceptación.

3.2 Estado del producto al inicio de EP3

Al iniciar EP3 (semana 11), el sistema contaba con los siguientes componentes estabilizados desde EP2:

- **Sistema RBAC/JWT:** completo y estable. Roles, permisos y autenticación operativos en producción.

- **Módulo territorial básico:** mapa choropleth con capas NDVI/NDMI/FIRMS/ALERTS/REPORTS disponibles como toggles, pero sin panel comunal de detalle ni score WLC integrado.
- **Score WLC (Weighted Linear Combination):** implementado en el backend en modos STANDARD y ENHANCED, pero sin exposición completa en el frontend.
- **Módulo comunitario parcial:** tablero de publicaciones y biblioteca de recursos funcionando. Agenda de contactos con bug de persistencia (comunal ausente en payload).
- **Panel analítico regional:** visible como ruta /dashboard independiente, sin sincronización automática con la región seleccionada en el mapa.
- **Módulo de alertas:** reglas configurables creadas, pero sin indicadores de ayuda para umbrales ni toggle de monitoreo por región.

3.3 Arquitectura del sistema (al cierre de EP3)

Stack completo

Capa	Tecnología	Plataforma de despliegue
Frontend	React 18 + Vite + JSX	Vercel
Backend API	Java 21 + Spring Boot 3	Railway
Servicio analítico satelital	Python 3.11 + FastAPI + OpenEO	Railway
Base de datos relacional	PostgreSQL 15	Supabase
Base de datos documental	MongoDB Atlas	Atlas Cloud (M0)

Flujo de datos

El sistema consume datos de tres categorías de fuentes externas:

1. **NASA FIRMS (VIIRS)**: focos de calor activos sincronizados dos veces al día (00:00 y 12:00). El backend consume la API REST de NASA Earthdata directamente en Java.
2. **Open-Meteo / FWI**: datos meteorológicos incluyendo Fire Weather Index (FWI), temperatura aire y suelo, humedad relativa y viento. Sincronización dos veces al día (00:30 y 12:30).
3. **Copernicus/OpenEO**: índices de vegetación NDVI (Normalized Difference Vegetation Index) y humedad NDMI (Normalized Difference Moisture Index). Sincronización on-demand por solicitud del usuario desde el panel comunal, o en ciclo programado diario.

El pipeline de procesamiento:

Fuentes externas → Cron jobs (backend Java) → MongoDB Atlas



TerritoryRiskService

calcula score WLC



ComunaRiskScore → RegionalRiskScore



Evaluación de AlertRules



Alert + Notification generadas



Frontend (React) ←← API REST (Spring Boot) ←← Consultas por regionId

Programación de cron jobs

Job	Horario	Función
NasaFirmsSync	00:00 y 12:00	Sincroniza focos FIRMS desde NASA Earthdata
OpenMeteoFwiSync	00:30 y 12:30	Sincroniza FWI y datos meteorológicos
RegionalRiskCalc	01:00 diario	Calcula RegionalRiskScore a partir de ComunaRiskScores
ComunaRiskCalc	01:30 y 13:30	Recalcula ComunaRiskScore por todas las regiones con monitoringEnabled

Score WLC (Weighted Linear Combination)

El score de riesgo es el indicador central del sistema. Se calcula como una combinación ponderada de indicadores normalizados al rango [0, 1]:

Modo STANDARD (sin confirmación satelital Copernicus):

Variable	Peso
FWI (Fire Weather Index)	52%
FIRMS (focos activos)	33%
Reportes ciudadanos	15%

Modo ENHANCED (con datos Copernicus/OpenEO disponibles):

Variable	Peso
FWI (Fire Weather Index)	38%
NDMI (humedad de vegetación)	22%
FIRMS (focos activos)	18%

Variable	Peso
NDVI (índice de vegetación)	8%
Reportes ciudadanos	4%

El score compuesto se almacena en la colección comunaRiskScores junto con el desglose por componente, y se actualiza automáticamente en cada ciclo de sync o al recibir una confirmación Copernicus manual.

Los niveles de alerta derivados del score son:

Nivel	Descripción
NORMAL	Condiciones dentro de parámetros históricos esperados
PREVENTIVO	Indicadores en zona de atención, monitoreo intensificado recomendado
ALTO	Condiciones de riesgo significativo, activación de protocolos preventivos
CRÍTICO	Riesgo extremo, requiere respuesta inmediata

3.4 Módulos implementados en EP3

3.4.1 Módulo Territorial

El módulo territorial es el núcleo operativo de SIMFAT. Al cierre de EP3 incluye:

- **Mapa choropleth interactivo:** visualización de comunas coloreadas según nivel de riesgo (NORMAL/PREVENTIVO/ALTO/CRÍTICO) usando polígonos GADM simplificados, renderizados con Leaflet y React-Leaflet. La selección de región actualiza todos los elementos del mapa y el panel analítico embebido.
- **Panel comunal de detalle:** al hacer clic en una comuna, se despliega un panel lateral con el score WLC compuesto, el nivel de alerta, el desglose por componente (contribución de FWI, NDMI, NDVI, FIRMS y reportes en porcentaje del score total) y el nombre de la comuna en el encabezado.
- **Confirmación con Copernicus:** botón "Confirmar con Copernicus" en el panel comunal que solicita al openeo-service un sync satelital on-demand.

El proceso tarda entre 70 y 90 segundos y al completarse transiciona el modo de STANDARD a ENHANCED con datos actualizados de NDVI y NDMI.

- **Capas de mapa activables por toggle:** FIRMS (focos de calor), NDVI, NDMI, Viento (flechas de dirección con tooltip de velocidad en km/h), Alertas y Reportes ciudadanos.
- **Capa de viento:** flechas orientadas según datos de Open-Meteo, con color según velocidad y tooltip que muestra velocidad y punto cardinal. Filtradas por bounding box de la región activa.
- **Filtrado FIRMS corregido:** en EP3 se corrigió el bug por el que el conteo de focos FIRMS mostraba el total global en lugar de los focos de la región activa. La corrección aplica un filtro client-side por bounding box del GeoJSON comunal de la región seleccionada.
- **Exportación de informes PDF:** mediante window.print() nativo del navegador, sin dependencias externas. Disponible para informes regionales desde el panel analítico y para informes comunales desde el panel de detalle de cada comuna.

3.4.2 Panel Analítico Regional (Dashboard)

El panel analítico regional fue refactorizado en EP3: se eliminó la ruta /dashboard del menú de navegación y el panel se integró directamente en TerritoryPage, sincronizándose automáticamente con la región seleccionada en el mapa.

Indicadores disponibles en el panel:

- **KPIs de riesgo:** focos FIRMS filtrados por región (conteo corregido), alertas activas (comunas en ALTO + comunas en CRÍTICO), reportes ciudadanos activos en la región.
- **Vegetación satelital:** NDVI y NDMI promedio regional con escala de interpretación (verde: saludable, rojo: degradado).
- **Datos meteorológicos:** velocidad del viento, humedad relativa, temperatura del aire y temperatura del suelo, obtenidos de Open-Meteo.
- **Exportación:** botón de informe regional desde el footer del panel, que genera una ventana de impresión con los KPIs e indicadores de la región activa.

3.4.3 Módulo de Alertas y Reglas

El módulo de alertas permite a los administradores configurar reglas de alerta con umbrales específicos por indicador y región, y visualizar las alertas generadas automáticamente por el sistema.

Gestión de reglas (RulesPage):

- CRUD completo: crear, editar y eliminar reglas de alerta con modal de confirmación para eliminación.
- Umbrales configurables para: FWI, NDMI, NDVI, recuento de focos FIRMS y recuento de reportes ciudadanos.
- Mejora EP3: textos de ayuda en el formulario que muestran los rangos típicos y críticos para cada variable (FWI: 0–50+, NDMI: -1 a 1, NDVI: 0–1, etc.), reduciendo el riesgo de configurar reglas con umbrales sin significado.

Alertas activas:

- Lista de alertas con nivel de severidad diferenciado por color (PREVENTIVO/ALTO/CRÍTICO).
- Filtros por región y nivel de riesgo.
- Mapa de alertas con geolocalización de las comunas afectadas.

Notificaciones:

- Campana en la barra de navegación con badge de conteo de notificaciones no leídas.
- Polling cada 30 segundos hacia el endpoint `/api/notifications`.
- Las notificaciones se generan como consecuencia de alertas activas (cuando el backend detecta que un indicador supera el umbral configurado en una `AlertRule`).

3.4.4 Módulo Comunitario

El módulo comunitario centraliza la coordinación entre la AIFBN y la red de contactos forestales en cada región.

Tablero de publicaciones:

- Publicaciones visibles para usuarios de la región asignada.
- Prioridad visual diferenciada: ALTA, MEDIA, BAJA.
- Formulario de nueva publicación disponible para usuarios con permisos.
- Ordenamiento cronológico descendente.

Biblioteca de recursos:

- Documentos, guías y protocolos forestales.
- Mejora EP3: restricción de carga a solo archivos PDF, con mensaje de validación al intentar subir otros formatos.
- Filtro por tipo de recurso.

Agenda de contactos:

- Listado de contactos con nombre, organización, teléfono, email, región y comuna.
- Mejora EP3: el campo comunald ahora se incluye en el payload de creación, corrigiendo el bug que causaba error HTTP 400 silencioso al guardar un contacto nuevo.
- Filtro de búsqueda en tiempo real por nombre o institución.

Chat comunitario:

- Salas de chat: sala GENERAL disponible por defecto más salas regionales.
- Sala GENERAL como fallback si el servicio de salas de la API devuelve lista vacía.
- Acceso controlado por el campo communityChatAccess del perfil de usuario y por la región asignada.

3.4.5 Panel de Accesos RBAC

El panel de control de accesos permite a los administradores gestionar usuarios y sus permisos.

Gestión de usuarios:

- Sistema de roles dual: legacy (enum MongoDB) y moderno (roleCodes en PostgreSQL).
- Perfiles predefinidos con conjuntos de roles: COMMUNITY, VERIFIED, MODERATOR, ADMIN, SUPER_ADMIN.
- Asignación de acceso al chat comunitario por región desde el perfil del usuario.

Estados de verificación de identidad:

- EMAIL_VERIFIED: email confirmado mediante token de verificación.

- PHONE_VERIFIED: número de teléfono verificado.
- IDENTITY_VERIFIED: identidad formal validada por un administrador.
- SUSPENDED: cuenta suspendida temporalmente.
- Historial de eventos de verificación por usuario con timestamps y responsable de la acción.

Administración de regiones:

- Toggle de monitoreo activo/inactivo por región (monitoringEnabled).
- Al desactivar el monitoreo de una región, los cron jobs la excluyen de los ciclos de sync, optimizando el uso de cuotas de APIs externas.

4. Plan de Pruebas de Software (EP3)

El plan de pruebas de EP3 cubre las funcionalidades implementadas hasta el cierre del período, organizado en tres categorías: pruebas funcionales por caso de uso (CU), pruebas de seguridad (SEC) y pruebas de usabilidad (USAB). El total de casos definidos es 34.

Plan de Pruebas de Software — EP3

SIMFAT — Sistema de Monitorización Forestal y Análisis Territorial

- **Curso:** TPY1101 – Taller Aplicado de Programación
- **Institución:** Duoc UC
- **Estudiante:** David Vásquez
- **Fecha:** 2026-06-20
- **Versión:** 1.0
- **Ambiente de prueba:** Producción (simfat-web en Vercel + backend Railway)

Pruebas funcionales — Casos de uso

ID	Funcionalidad	Acción de prueba	Resultado esperado	Resultado obtenido	Estado
C U0 1- T0 1	Login / autenticación JWT	Ingresar email y contraseña válidos en /login y presionar "Ingresar"	Se genera token JWT, se redirige a /territorio y el nombre del usuario aparece en la barra lateral	El sistema autentica correctamente, almacena el token en localStorage y redirige al mapa territorial. El token incluye rol y expiración.	APROBADO
C U0 1- T0 2	Login / autenticación JWT	Ingresar credenciales incorrectas	Mensaje de error claro ("Credenciales inválidas"), sin redirección, sin token emitido	Se muestra mensaje de error del backend; el formulario permanece activo sin limpiar el email	APROBADO
C U0 1- T0 3	Login — token persistente	Cerrar pestaña y reabrir la aplicación con sesión activa	La sesión se mantiene activa sin requerir nuevo login (token no expirado)	La sesión persiste correctamente al reabrir la pestaña dentro del tiempo de validez del token	APROBADO

ID	Funcionalidad	Acción de prueba	Resultado esperado	Resultado obtenido	Estado
C U0 2- T0 1	Mapa territorial — choropleth de riesgo	Seleccionar región "Araucanía" en el selector y visualizar el mapa	El mapa muestra comunas coloreadas según nivel de riesgo (NORMAL/PREVENTIVO/ALTO/CRÍTICO) con leyenda visible	Las comunas se colorean con la paleta de riesgo definida. La leyenda aparece en la esquina inferior del mapa. La capa de choropleth usa polígonos GADM correctamente simplificados.	APROBADO
C U0 2- T0 2	Mapa territorial — selección de comuna	Hacer clic sobre una comuna en el mapa	Aparece panel lateral con score WLC, nivel de alerta, y desglose de componentes	El panel comunal se abre con los datos de la comuna seleccionada: score compuesto, nivel de alerta, y contribuciones de NDVI, NDMI, FWI, FIRMS y reportes ciudadanos	APROBADO

ID	Funcionalidad	Acción de prueba	Resultado esperado	Resultado obtenido	Estado
C U0 2- T0 3	Mapa territorial — capa FIRMS	Activar toggle "FIRMS" en la barra de controles	Aparecen puntos de fuego sobre el mapa correspondientes a la región seleccionada	Los focos FIRMS aparecen como marcadores. El conteo muestra solo focos dentro del bounding box de la región activa (filtro client-side corregido en este sprint)	APROBADO
C U0 2- T0 4	Mapa territorial — capa de viento	Activar toggle "Viento" con región Araucanía seleccionada	Aparecen flechas de dirección de viento por comuna con color según velocidad	Las flechas aparecen orientadas según datos de Open-Meteo. El tooltip muestra velocidad (km/h) y punto cardinal. El color varía según la escala de la leyenda.	APROBADO
C U0	Panel analítico	Seleccionar una	El panel muestra KPIs regionales:	El dashboard analítico se	APROBADO

ID	Funcionalidad	Acción de prueba	Resultado esperado	Resultado obtenido	Estado
3-T01	o regional	región y desplazar se al panel de indicadores en TerritoryPage	FWI promedio, NDVI, NDMI, focos FIRMS, alertas activas, reportes ciudadanos	embebe correctamente en TerritoryPage y se actualiza automáticamente al cambiar la región seleccionada	
C U03-T02	Exportar informe regional al PDF	Hacer clic en "Exportar informe regional" en el panel analítico	Se abre ventana de impresión con el informe regional formateado, listo para guardar como PDF	La ventana de impresión del navegador se abre con el informe incluyendo nombre de región, fecha, KPIs y gráficos	APROBADO
C U04-T01	Panel comunal — score WLC	Seleccionar una comuna y revisar el desglose del score	Se muestra el score WLC compuesto (0–100) con la contribución de cada variable: FWI, NDMI, NDVI, FIRMS, reportes	El panel muestra el score compuesto y los pesos de cada componente correctamente. El nombre de la comuna aparece en el	APROBADO

ID	Funcionalidad	Acción de prueba	Resultado esperado	Resultado obtenido	Estado
				encabezado del panel.	
C U0 4- T0 2	Panel comunal — confirmación Copernicus	Hacer clic en "Confirmar con Copernicus" en el panel comunal	Se inicia el sync OpenEO (~70s) y el resultado muestra modo ENHANCED con NDVI/NDMI satelitales actualizados	El proceso de sync se ejecuta, el indicador de carga aparece, y al completarse el panel muestra datos ENHANCED. El botón de impresión permanece accesible.	APROBADO
C U0 5- T0 1	Reglas de alerta — crear	Acceder a /admin/rules y crear una regla con umbral FWI = 20 para una región	La regla se guarda y aparece en la tabla de reglas activas	La regla se crea correctamente y aparece en la tabla con los umbrales configurados (FWI, NDMI, NDVI, FIRMS, reportes) resumidos en una columna	APROBADO

ID	Funcionalidad	Acción de prueba	Resultado esperado	Resultado obtenido	Estado
C U0 5- T0 2	Reglas de alerta — editar	Editar una regla existente y cambiar el umbral de reportes ciudadanos	Los cambios se persisten y la tabla refleja el nuevo valor	La edición se guarda sin errores. El formulario pre-carga los valores existentes correctamente.	APROBADO
C U0 5- T0 3	Reglas de alerta — eliminar	Eliminar una regla usando el botón de eliminación con modal de confirmación	La regla desaparece de la tabla tras confirmar la eliminación	El modal de confirmación aparece antes de eliminar. Tras confirmar, la regla se elimina y la tabla se actualiza.	APROBADO
C U0 6- T0 1	Alertas activas — visualización	Acceder a /alertas y revisar las alertas generadas	Se muestra lista de alertas con nivel (PREVENTIVO/ALTO/CRÍTICO), región, mensaje y fecha	Las alertas aparecen con sus niveles diferenciados por color. Los filtros de región y nivel funcionan correctamente.	APROBADO
C U0 7-	Tablero comun	Acceder a /comunidad y	Se muestran publicaciones del tablero ordenadas	Las publicaciones aparecen con	APROBADO

ID	Funcionalidad	Acción de prueba	Resultado esperado	Resultado obtenido	Estado
T01	itario — publicaciones	revisar la sección de publicaciones	por fecha, con prioridad visual diferenciada	título, mensaje, prioridad y autor. El formulario de nueva publicación está disponible.	
C U08-T01	Biblioteca de recursos — listar	Acceder a la sección de recursos dentro de /comunidad	Se muestra la lista de documentos/recursos disponibles para la comunidad	La biblioteca muestra los recursos cargados. El filtro por tipo funciona.	APROBADO
C U08-T02	Biblioteca de recursos — cargar PDF	Intentar subir un archivo que no sea PDF	El sistema rechaza el archivo y muestra un mensaje de validación	Solo se permiten archivos PDF. Se muestra mensaje de validación cuando se intenta subir otro formato (restricción implementada en este sprint).	APROBADO

ID	Funcionalidad	Acción de prueba	Resultado esperado	Resultado obtenido	Estado
C U0 9- T0 1	Agenda de contactos — crear	Agregar un nuevo contacto con nombre, organización, teléfono, email, región y comuna	El contacto se guarda y aparece en la lista de contactos	El formulario incluye el campo comunal. El payload enviado al backend incluye comunal, evitando el error 400 anterior. El contacto aparece en la agenda con la información correcta.	APROBADO
C U0 9- T0 2	Agenda de contactos — buscar	Usar el filtro de búsqueda para encontrar un contacto por nombre o institución	Los resultados se filtran dinámicamente mostrando solo los contactos que coinciden	El filtro funciona en tiempo real sobre la lista cargada.	APROBADO
C U1 0-	Chat comunitario —	Enviar un mensaje en el chat de la	El mensaje aparece en la conversación con nombre del	El chat muestra el mensaje enviado. La	PENDIENTE

ID	Funcionalidad	Acción de prueba	Resultado esperado	Resultado obtenido	Estado
T01	enviar mensaje	comunidad	usuario y hora de envío	conectividad en tiempo real depende del servicio WebSocket disponible en Railway.	
C U11-T01	Notificaciones — recibir	Ejecutar acción que dispara una alerta (ej. superar umbral FWI configurado)	El usuario recibe una notificación visible en la barra de la aplicación	Las notificaciones se generan cuando las reglas de alerta se activan. El badge de notificaciones se actualiza.	PENDIENTE
C U12-T01	Exportar informe regional PDF	Hacer clic en "Exportar informe" desde el panel regional	Se genera ventana de impresión con el informe regional completo en formato legible	El informe incluye nombre de región, KPIs, fecha y datos de indicadores. La ventana de impresión del navegador se abre automáticamente sin	APROBADO

ID	Funcionalidad	Acción de prueba	Resultado esperado	Resultado obtenido	Estado
				dependencias externas.	
C U13-T01	Exportar informe comunal PDF	Seleccionar una comuna y hacer clic en el botón de impresión del panel comunal	Se genera ventana de impresión con el informe comunal incluyendo score WLC y desglose por indicador	El informe comunal muestra el nombre de la comuna, score compuesto, nivel de alerta y contribuciones por variable. El botón de impresión se ubica en posición accesible dentro del panel.	APROBADO
C U14-T01	Panel de accesos RBAC — visualizar usuarios	Acceder a /admin/access-control con cuenta ADMIN	Se muestra la lista de usuarios con sus roles asignados	La tabla de usuarios carga correctamente con nombre, email y rol. Las acciones de gestión de roles están disponibles solo para	OBSERVADO

ID	Funcionalidad	Acción de prueba	Resultado esperado	Resultado obtenido	Estado
				cuentas con los permisos adecuados.	
C U15-T01	Administración de regiones — listar	Acceder a /admin/regions con cuenta ADMIN	Se muestra la lista de regiones con su estado de monitoreo	La tabla muestra las regiones registradas con el toggle de monitoreo activo/inactivo por región.	APROBADO
C U15-T02	Administración de regiones — toggle monitoreo	Activar o desactivar el monitoreo de una región usando el toggle	El estado de monitoreo se persiste y afecta si la región recibe sync programado	El toggle cambia el campo monitoringEnabled en el backend. La región deja de recibir actualizaciones de sync si se desactiva.	APROBADO

Pruebas de seguridad

ID	Funcionalidad	Acción de prueba	Resultado esperado	Resultado obtenido	Estado
SEC01-	Protección de rutas	Acceder directamente	El sistema redirige automática	ProtectedRoute intercepta la	APROBADO

ID	Funcionalidad	Acción de prueba	Resultado esperado	Resultado obtenido	Estado
T01	— ruta protegida sin sesión	a /territorio sin estar autenticado	mente a /login	navegación y redirige a /login. No se expone contenido protegido.	
SEC01-T02	Protección de rutas — ruta pública con sesión activa	Acceder a /login con sesión JWT activa	El sistema redirige automáticamente al módulo principal /territorio	PublicOnlyRoute detecta el token activo y redirige. La página de login no es accesible con sesión iniciada.	APROBADO
SEC01-T03	Protección de rutas admin — usuario sin rol ADMIN	Acceder a /admin/rules con una cuenta de usuario normal	El sistema muestra un mensaje de acceso denegado o redirige sin mostrar el contenido	Las rutas /admin/* verifican el rol del usuario. Un usuario sin ROLE_ADMIN no puede ver ni interactuar con las páginas de administración.	APROBADO

ID	Funcionalidad	Acción de prueba	Resultado esperado	Resultado obtenido	Estado
SEC02-T01	Validación de inputs — campo de texto	Ingresar <code><script>alert('xss')</script></code> en el campo de mensaje del chat o de publicación	El texto se muestra como literal, sin ejecutar el script	React escapa el contenido por defecto al renderizar en el DOM. No se ejecutan scripts inyectados en campos de texto.	APROBADO
SEC02-T02	Validación de inputs — campos numéricos de reglas	Ingresar un valor negativo o un string no numérico en los umbrales de una <code>AlertRule</code>	El backend rechaza la solicitud con error de validación (400)	El backend valida los tipos. El frontend muestra los mensajes de error de validación retornados por el servidor.	APROBADO
SEC03-T01	Control de roles — botón de sync clima	Verificar visibilidad del botón "Sincronizar clima y viento" con distintos roles	Solo visible para <code>ROLE_ADMIN</code> y <code>ROLE_SUPER_ADMIN</code> ; oculto para otros roles (no solo	El botón no se renderiza para usuarios sin el rol requerido. No aparece en el DOM para cuentas no	APROBADO

ID	Funcionalidad	Acción de prueba	Resultado esperado	Resultado obtenido	Estado
			deshabilitado)	administrativas.	
SEC03-T02	Control de roles — acceso a panel de accesos	Verificar que /admin/accesos-control no es accesible para ROLE_COMMUNITY_USER	La ruta no carga para usuarios sin permisos; se redirige o muestra acceso denegado	La protección de rutas por rol impide el acceso. Confirmado en integración con el sistema RBAC.	OBSE RVADO

Pruebas de usabilidad

ID	Funcionalidad	Acción de prueba	Resultado esperado	Resultado obtenido	Estado
USAB01-T01	Responsive design — mobile	Abrir la aplicación en un viewport de 375×667 px (iPhone SE)	El layout se adapta: sidebar colapsa, el mapa es navegable, los formularios son legibles	El layout principal usa clases responsivas. La sidebar colapsa en viewports pequeños. El mapa Leaflet es funcional en móvil	APROBADO

ID	Funcionalidad	Acción de prueba	Resultado esperado	Resultado obtenido	Estado
				con controles accesibles.	
USA B01-T02	Responsividad design — tablet	Abrir la aplicación en un viewport de 768×1024 px (iPad)	El contenido usa el espacio horizontal disponible; los paneles laterales no se superponen	El layout intermedio se muestra correctamente en resolución tablet. Los paneles del mapa territorial mantienen proporciones adecuadas.	APROBADO
USA B02-T01	Accesibilidad — contraste de colores	Revisar el tooltip del choropleth con diferentes niveles de riesgo	El texto del tooltip es legible sobre todos los fondos de color de la escala de riesgo	El tooltip usa color de texto oscuro para fondos claros y darkUiColor para fondos oscuros, aplicando la lógica de contraste corregida	APROBADO

ID	Funcionalidad	Acción de prueba	Resultado esperado	Resultado obtenido	Estado
				en este sprint.	
USA B02-T02	Accesibilidad — semántica HTML	Inspeccionar el DOM de la barra de navegación y los formularios	Los elementos usan etiquetas semánticas (nav, main, button, label) y los campos tienen atributos for/id vinculados	La estructura HTML usa etiquetas semánticas. Los formularios tienen labels asociados a sus inputs. Los botones usan <button> y no <div> clickable.	APROBADO
USA B02-T03	Accesibilidad — escala colorblind-safe	Revisar la leyenda del choropleth y las escalas de clima/vegetación	Los colores usados son distinguibles para usuarios con deuteranopia/protanopia (paleta colorblind-safe)	Las escalas de color fueron refactorizadas a paletas colorblind-safe (naranja/azul en lugar de rojo/verde) en el sprint	APROBADO

ID	Funcion alidad	Acción de prueba	Resultado esperado	Resultado obtenido	Estado
				anterior. La leyenda refleja estos colores.	

Resumen de estados del plan de pruebas

Estado	Cantidad
APROBADO	30
OBSERVADO	2
PENDIENTE	2
Total	34

Casos OBSERVADO:

- **CU14-T01:** La gestión de roles en /admin/access-control carga correctamente, pero la confirmación de QA con usuario real de producción está pendiente de ejecución formal con Andrés.
- **SEC03-T02:** La protección de ruta /admin/access-control para ROLE_COMMUNITY_USER está implementada a nivel de componente, pero falta evidencia de ejecución en producción con ese rol específico.

Casos PENDIENTE:

- **CU10-T01 (Chat):** Depende de conectividad WebSocket con el servicio Railway. La prueba requiere un ambiente con dos usuarios simultáneos conectados.

- **CU11-T01 (Notificaciones push):** La generación automática de notificaciones al disparar una regla requiere que el scheduler esté activo en Railway y que haya datos reales que superen el umbral configurado. No es reproducible de forma inmediata en prueba manual.

4.1 Descripción del ambiente de pruebas

- **Ambiente de producción:** Vercel (frontend) + Railway (backend + openeo-service)
- **Ambiente local:** localhost:5173 (frontend Vite) + localhost:8080 (Spring Boot)
- **Base de datos de pruebas:** MongoDB Atlas (colección simfat) + PostgreSQL Supabase
- **Usuarios de prueba con rol ADMIN:** jennifer@aifbn.cl, pablo@aifbn.cl
- **Herramientas de apoyo:** Swagger UI (contratos API en <https://simfat-backend-production.up.railway.app/swagger-ui/index.html>), Chrome DevTools (red/console), Postman (validación de endpoints)

5. Pruebas de Validación Aplicadas

5.1 Resultados de validación por componente

Componente: Autenticación JWT (CU01)

Las tres pruebas del componente de autenticación resultaron APROBADO. El token JWT se genera correctamente al autenticar con credenciales válidas e incluye el rol y la fecha de expiración. La sesión persiste en localStorage con el TTL correcto configurado en application.properties. El mensaje de error es apropiado ante credenciales inválidas y el formulario no se limpia, facilitando la corrección del campo erróneo por parte del usuario.

Componente: Mapa territorial (CU02)

Las cuatro pruebas del mapa territorial resultaron APROBADO. El choropleth renderiza correctamente las comunas con los colores de riesgo correspondientes a cada nivel. El panel comunal se abre con todos los datos del score WLC al hacer clic en una comuna. La capa FIRMS muestra únicamente los focos correspondientes a la región activa (corrección aplicada en EP3). La capa de viento muestra flechas orientadas con tooltip de velocidad y punto cardinal.

Componente: Panel analítico (CU03)

Ambas pruebas APROBADO. El dashboard analítico se integra correctamente en TerritoryPage y responde automáticamente al cambio de región en el mapa. La exportación de informe regional mediante `window.print()` funciona sin dependencias externas.

Componente: Panel comunal WLC (CU04)

Ambas pruebas APROBADO. El score compuesto se muestra con el desglose correcto por componente. La confirmación con Copernicus completa el ciclo en aproximadamente 70-90 segundos y transiciona el modo del panel de STANDARD a ENHANCED.

Componente: Reglas de alerta (CU05)

Las tres pruebas del CRUD de reglas resultaron APROBADO. Las validaciones frontend y backend funcionan correctamente. El formulario precarga los valores existentes al editar y el modal de confirmación previene eliminaciones accidentales.

Componente: Alertas activas (CU06)

Prueba APROBADO. Los filtros de región y nivel de riesgo funcionan correctamente y los niveles se diferencian visualmente por color.

Componente: Módulo comunitario (CU07-CU09)

Las pruebas CU07, CU08 y CU09 resultaron APROBADO. El tablero de publicaciones, la biblioteca de recursos y la agenda de contactos funcionan correctamente. La restricción de solo PDF en la biblioteca está activa. El campo comunal en el payload de creación de contactos corrige el bug de persistencia identificado durante QA.

Componente: Chat comunitario (CU10)

Estado PENDIENTE. La funcionalidad de chat está implementada en frontend y backend, pero la prueba formal requiere conectividad WebSocket activa entre dos usuarios simultáneos en Railway, condición que no fue reproducible de forma controlada durante el período de QA.

Componente: Notificaciones (CU11)

Estado PENDIENTE. La infraestructura de notificaciones está implementada (colección notifications, polling de 30 segundos en el frontend, badge en navbar), pero la generación automática de notificaciones depende del cron job de evaluación de AlertRules ejecutándose con datos reales que superen los umbrales configurados en el ambiente de Railway.

Componente: Informes PDF (CU12-CU13)

Ambas pruebas APROBADO. Los informes regionales y comunales se generan correctamente con datos actualizados. El botón de impresión en el panel comunal fue reubicado en EP3 para evitar superposición con el contenido del desglose.

Componente: RBAC y accesos (CU14-CU15)

CU14 OBSERVADO: el panel de gestión de usuarios carga correctamente y las acciones de gestión de roles están disponibles, pero la confirmación formal de QA con un usuario real de producción en el rol ADMIN está pendiente. CU15 APROBADO en ambas sub-pruebas: la lista de regiones carga correctamente y el toggle de monitoreo persiste el cambio en el backend.

Pruebas de seguridad (SEC01-SEC03)

- SEC01-T01, T02, T03: todos APROBADO. ProtectedRoute y PublicOnlyRoute interceptan correctamente la navegación no autorizada.
- SEC02-T01: APROBADO. React escapa el contenido HTML por defecto, previniendo inyección XSS.
- SEC02-T02: APROBADO. El backend retorna errores 400 estructurados ante inputs inválidos.
- SEC03-T01: APROBADO. El botón de sync de clima no se renderiza para roles sin permisos.
- SEC03-T02: OBSERVADO. La protección de ruta está implementada pero falta evidencia de ejecución en producción con ROLE_COMMUNITY_USER.

Pruebas de usabilidad (USAB01-USAB02)

Las cinco pruebas de usabilidad resultaron APROBADO. El layout es responsivo en móvil y tablet. El contraste del tooltip del choropleth fue corregido en EP3 usando darkUiColor para el nivel CRÍTICO. Las escalas de color colorblind-safe aplicadas en el sprint anterior están activas en el choropleth y las capas de clima y vegetación.

5.2 Resumen de resultados

Estado	Cantidad	Porcentaje
APROBADO	30	88.2%
OBSERVADO	2	5.9%
PENDIENTE	2	5.9%
Total	34	100%

Los 2 casos OBSERVADO requieren validación con usuarios reales en producción pero no bloquean funcionalidad — la implementación está completa. Los 2 casos PENDIENTE dependen de condiciones de entorno no disponibles en prueba manual controlada (WebSocket activo para chat y cron job con datos reales para notificaciones).

6. Mejoras Implementadas EP3

Las mejoras implementadas en EP3 responden directamente al ciclo de pruebas y retroalimentación del equipo QA (Andrés Ibáñez). Cada mejora está trazada a uno o más casos de prueba del plan formal.

Mejoras Implementadas — Evaluación Parcial N°3 (EP3)

SIMFAT — Sistema de Monitorización Forestal y Análisis Territorial

- **Curso:** TPY1101 – Taller Aplicado de Programación
- **Institución:** Duoc UC
- **Estudiante:** David Vásquez
- **Fecha de sprint:** 2026-06-19 / 2026-06-20
- **Período evaluado:** EP3

Tabla de mejoras

#	Categoría	Mejora	Módulo afectado	Impacto
1	Corrección	Filtro client-side de focos FIRMS por región: el conteo de focos activos ahora filtra por el bounding box del GeoJSON comunal de la región seleccionada, eliminando el bug que mostraba el total global de detecciones en lugar de las de la región activa.	Territorio / DashboardPage.tsx, hook useTerritoryLayers	El indicador "Focos FIRMS" en el panel analítico regional ahora refleja datos pertinentes a la región visualizada. Elimina una discrepancia significativa en la información mostrada al usuario.
2	Complejidad	Dashboard analítico regional embebido en TerritoryPage: se eliminó la ruta /dashboard del menú de módulos y el panel se	Territorio / TerritoryPage, AppRouter.jsx, navigationConfig.js	El flujo de análisis es ahora continuo: el coordinador

#	Categoría	Mejora	Módulo afectado	Impacto
		integró directamente en la página de territorio, actualizándose automáticamente al cambiar la región seleccionada en el mapa.		selecciona una región en el mapa y los indicadores analíticos responden sin necesidad de navegar a otra pantalla.
3	Complejidad	Exportación de informes en PDF para nivel regional y comunal: implementación de ventana de impresión auto-generada desde el frontend usando la API nativa del navegador (<code>window.print()</code>), sin dependencias externas ni servidores de renderizado.	Territorio / <code>reportPrint.ts</code> (regional), panel comunal	Los coordinadores pueden generar y guardar informes PDF de cualquier región o comuna directamente desde el mapa, cubriendo los requisitos de exportación

#	Categoría	Mejora	Módulo afectado	Impacto
				ón sin infraestructura adicional.
4	Corrección	Persistencia de contactos comunitarios: el payload enviado al backend al crear un contacto ahora incluye comunald, campo requerido por el endpoint /api/community/contacts. La omisión causaba error HTTP 400 con silencio en UI.	Comunidad / formulario de contactos	Los contactos se guardan correctamente en la agenda. Elimina un bug de regresión que impedía la creación de nuevos registros de contacto comunitario.
5	Usabilidad	Contraste del tooltip del choropleth mejorado: la lógica de color de texto del tooltip ahora elige entre texto	Territorio / componente de tooltip del mapa	Mejora la legibilidad del tooltip para todos los niveles

#	Categoría	Mejora	Módulo afectado	Impacto
		oscuro o claro según la luminosidad del color de fondo del nivel de alerta, usando darkUiColor para fondos oscuros (CRÍTICO) y color negro para fondos claros (NORMAL/PREVENTIVO).		de riesgo, especialmente en el nivel CRÍTICO donde el texto oscuro sobre fondo rojo oscuro era ilegible.
6	Corrección	Panel de informe comunal: corregidos tres problemas en el panel de detalle de comuna: (a) el nombre de la comuna ahora aparece en el encabezado, (b) el botón de impresión fue reubicado a una posición accesible y no superpuesta con el contenido, y (c) la contribución de cada componente al score WLC se calcula	Territorio / panel comunal	El informe comunal es ahora informativo y accionable. La corrección del cálculo de contribución elimina valores incorrectos en el desglose

#	Categoría	Mejora	Módulo afectado	Impacto
		correctamente usando component .score * component.weight en lugar de un valor plano.		del score.
7	Complejidad	Rangos sugeridos por indicador en el formulario de reglas de alerta: se agregaron textos de ayuda en el formulario de RulesPage que muestran los rangos típicos y críticos para cada variable configurable (FWI: 0–50+, NDMI: -1 a 1, NDVI: 0–1, FIRMS: conteo absoluto, reportes: conteo absoluto).	Admin / RulesPage.jsx	Permite al administrador configurar umbrales significativos sin necesidad de conocimiento previo del rango de cada índice, reduciendo el riesgo de configurar reglas que nunca se disparan o que se disparan siempre.

#	Categoría	Mejora	Módulo afectado	Impacto
8	Complejidad	Toggle de monitoreo activo en administración de regiones: la tabla de RegionsPage incluye un interruptor por región que activa o desactiva el flag monitoringEnabled, controlando si la región participa en los ciclos de sync programados (FIRMS, FWI, OpenEO).	Admin / RegionsPage.jsx, endpoint /api/admin/regions/{id}	Permite al administrador pausar el monitoreo de regiones que no requieren atención sin eliminar su configuración, optimizando el uso de las cuotas de APIs externas.
9	Pertinencia	Ruta /dashboard removida del menú de navegación: la ruta /dashboard existe como fallback pero ya no aparece en navigationConfig.js ni en la barra lateral. El acceso al panel analítico es ahora	Navegación / navigationConfig.js, AppRouter.jsx	Elimina la duplicidad de vistas que causaba confusión: los usuarios tenían dos

#	Categoría	Mejora	Módulo afectado	Impacto
		exclusivamente a través de la página de territorio al seleccionar una región.		lugares para ver indicadores regionales con datos potencialmente desincronizados. El flujo es ahora coherente con el modelo mental de "selección región → veo sus datos".

Trazabilidad con el plan de pruebas

Mejora #	Caso de prueba relacionado
1	CU02-T03 (capa FIRMS)
2	CU03-T01 (panel analítico regional)

Mejora #	Caso de prueba relacionado
3	CU12-T01 (informe regional PDF), CU13-T01 (informe comunal PDF)
4	CU09-T01 (agenda de contactos)
5	USAB02-T01 (contraste de colores)
6	CU04-T01 (panel comunal WLC), CU13-T01 (informe comunal PDF)
7	CU05-T01 (crear regla de alerta)
8	CU15-T02 (toggle monitoreo)
9	Mejora de navegación — sin caso de prueba funcional directo

Las 9 mejoras implementadas cubren todas las dimensiones de calidad requeridas por la rúbrica EP3: Corrección (bugs de filtro FIRMS, persistencia de contactos y cálculo de score comunal), Completitud (dashboard embebido, exportación PDF, rangos de umbrales y toggle de monitoreo), Usabilidad (contraste del tooltip) y Pertinencia (consolidación del flujo de navegación). Cada mejora está trazada a un caso de prueba específico del plan formal, lo que garantiza trazabilidad bidireccional entre detección del problema y resolución implementada.

7. Documentación Técnica

7.1 Diagrama de clases (EP3)

El diagrama de clases cubre las entidades principales del backend al cierre de EP3. Las entidades marcadas con <<PostgreSQL>> residen en la base de datos relacional; las demás entidades residen en MongoDB Atlas. Se muestran las relaciones conceptuales entre entidades de ambas bases de datos para representar el modelo completo del sistema.

El archivo fuente del diagrama se encuentra en Documentacion/UML/2026-06-20_diagrama_clases_ep3.md.

PostgreSQL (identidad y acceso)

```
class User {  
    <<PostgreSQL>>  
    +Long id  
    +String email  
    +String fullName  
    +String passwordHash  
    +Boolean verified  
    +LocalDateTime createdAt  
}
```

```
class UserRole {  
    <<PostgreSQL>>  
    +Long userId  
    +String roleCode  
}
```

```
class Role {  
    <<PostgreSQL>>  
    +String code  
    +String name  
    +String description  
}
```

%% — MongoDB (dominio territorial)

```
class Region {
    +String id
    +String nombre
    +String codigo
    +String zona
    +Double hectareasBosqueReferencia
    +Boolean monitoringEnabled
    +LocalDateTime updatedAt
}
```

```
class AlertRule {
    +String id
    +String nombre
    +String regionId
    +Double umbralFwi
    +Double umbralNdmi
    +Double umbralNdvi
    +Integer umbralFirmsCount
    +Integer umbralReportesCiudadanos
    +Boolean activa
    +LocalDateTime createdAt
}
```

```
class Alert {
```

```
+String id
+String level
+String regionId
+String comunalId
+String message
+String ruleId
+LocalDateTime createdAt
}
```

```
class Notification {
+String id
+String title
+String body
+String type
+Boolean read
+LocalDateTime createdAt
+String regionId
+Long userId
+String alertId
}
```

```
class ComunaRiskScore {
+String gadmGid
+String nombreComuna
+String regionId
+Double scoreComposite
+String alertLevel
}
```

```
+String mode
+Map components
+LocalDateTime computedAt
+LocalDateTime copernicusSyncedAt
}
```

```
class RegionalRiskScore {
+String regionId
+Double scoreComposite
+String alertLevel
+Integer comunasCount
+Integer comunasEnAlto
+Integer comunasEnCritico
+LocalDateTime generatedAt
}
```

%% — MongoDB (comunidad)

```
class CommunityPost {
+String id
+String title
+String message
+String priority
+String regionId
+LocalDateTime publishedAt
+String author
}
```

```
+Long authorUserId  
}
```

```
class CommunityContact {  
    +String id  
    +String name  
    +String organization  
    +String phone  
    +String email  
    +String regionId  
    +String comunalId  
    +String protocol  
    +LocalDateTime createdAt  
}
```

```
class CitizenReport {  
    +String id  
    +String category  
    +String description  
    +Double[] coordinates  
    +String regionId  
    +String comunalId  
    +LocalDateTime createdAt  
    +Long userId  
    +String status  
}
```


%% — Relaciones

User "1" ||--o{ UserRole : "tiene"

UserRole }o--|| Role : "asigna"

AlertRule }o--|| Region : "monitorea"

AlertRule "1" --o{ Alert : "dispara"

Alert }o--|| Region : "pertenece a"

Alert "1" --o{ Notification : "genera"

Notification }o--|| User : "notifica a"

ComunaRiskScore }o--|| Region : "pertenece a"

RegionalRiskScore }o--|| Region : "consolida"

CommunityPost }o--|| Region : "publicado en"

CommunityContact }o--|| Region : "asociado a"

CitizenReport }o--|| User : "enviado por"

CitizenReport }o--|| Region : "reportado en"

Descripción de entidades principales:

- **User / UserRole / Role (PostgreSQL):** Forman el núcleo del sistema de identidad y control de acceso. Un usuario puede tener múltiples roles, y cada rol agrupa un conjunto de permisos. Esta separación permite RBAC data-driven sin hardcode.

- **Region:** Unidad geográfica de monitoreo. El campo `monitoringEnabled` controla la participación de la región en los ciclos automáticos de sincronización.
- **AlertRule:** Regla de alerta configurada por un administrador para una región específica. Define umbrales por indicador. Cuando cualquier umbral es superado, el sistema genera una Alert.
- **Alert / Notification:** La alerta registra el evento de umbral superado; la notificación es el mensaje dirigido al usuario. Un evento de alerta puede generar múltiples notificaciones.
- **ComunaRiskScore:** Score WLC calculado por comuna. El campo `components` es un mapa que almacena el score parcial, el peso y el valor crudo de cada variable de riesgo. El campo `mode` indica si el cálculo incluye datos Copernicus (ENHANCED) o no (STANDARD).
- **RegionalRiskScore:** Agregación del riesgo comunal a nivel regional. Almacena el score consolidado y los conteos de comunas en ALTO y CRÍTICO para alimentar los KPIs del dashboard analítico.
- **CommunityContact:** Agenda de contactos forestales por región y comuna. El campo `comunalId` fue agregado en EP3 para corregir el bug de persistencia.

7.2 Modelo de base de datos (MER)

7.2.1 PostgreSQL – Identidad y acceso (RBAC)

El modelo de datos de PostgreSQL fue diseñado en EP2 siguiendo el contrato técnico RBAC documentado en `Documentacion/Informes/2026-05-14_fase0_rbac_jwt_contrato_arquitectura_v1.md`. Las tablas del modelo son:

Tabla	Propósito
users (APP_USERS)	Usuarios del sistema: email, nombre, hash de contraseña, estado de verificación
roles	Catálogo de roles disponibles: ROLE_SUPER_ADMIN, ROLE_ADMIN, ROLE_MODERATOR, ROLE_VERIFIED_USER, ROLE_COMMUNITY_USER

Tabla	Propósito
permissions	Catálogo de permisos por dominio funcional (ej. territory:read, community:write)
user_roles	Tabla de unión N:M entre usuarios y roles
role_permissions	Tabla de unión N:M entre roles y permisos
user_verification	Estado de verificación del usuario: EMAIL_VERIFIED, PHONE_VERIFIED, IDENTITY_VERIFIED, SUSPENDED
verification_events	Historial de cambios en el estado de verificación, con timestamp y responsable

Las migraciones de esquema se gestionan con Flyway y se aplican automáticamente al iniciar el backend.

7.2.2 MongoDB – Colecciones de negocio (nuevas en EP3)

El archivo fuente completo se encuentra en Documentacion/MER/2026-06-20_nuevas_colecciones_EP3.md.

Colección: alertRules

Propósito: Almacena las reglas de alerta configurables por región. Cada regla define uno o más umbrales para variables de riesgo. Cuando algún indicador supera su umbral, el sistema genera una Alert.

Campo	Tipo	Descripción
_id	ObjectId	Identificador único generado por MongoDB
nombre	String	Nombre descriptivo de la regla

Campo	Tipo	Descripción
regionId	String	Referencia a la región monitoreada (Region._id)
umbralFwi	Double	Umbral de Fire Weather Index; se dispara cuando FWI >= umbralFwi
umbralNdmi	Double	Umbral de NDMI; se dispara cuando NDMI <= umbralNdmi
umbralNdvi	Double	Umbral de NDVI; se dispara cuando NDVI <= umbralNdvi
umbralFirmsCount	Integer	Umbral de focos activos FIRMS; se dispara cuando count >= umbralFirmsCount
umbralReportesCiudadanos	Integer	Umbral de reportes ciudadanos activos
activa	Boolean	Indica si la regla está habilitada para evaluación
createdAt	DateTime	Fecha y hora de creación
updatedAt	DateTime	Fecha y hora de última modificación

Índices: { regionId: 1, activa: 1 } (compuesto).

Colección: alerts

Propósito: Registra las alertas generadas automáticamente cuando el sistema detecta que alguna regla de alerta ha sido superada.

Campo	Tipo	Descripción
_id	ObjectId	Identificador único
level	String (enum)	Nivel de severidad: PREVENTIVO, ALTO, CRÍTICO
regionId	String	Región donde se generó la alerta
comunalId	String	Comuna específica afectada (puede ser nulo para alertas regionales)
message	String	Mensaje descriptivo generado automáticamente
ruleId	String	Referencia a la AlertRule que disparó esta alerta
acknowledged	Boolean	Si un operador marcó la alerta como revisada
createdAt	DateTime	Fecha y hora de generación

Índices: { regionId: 1, createdAt: -1 } (compuesto), { level: 1 } (simple).

Colección: notifications

Propósito: Almacena las notificaciones dirigidas a usuarios específicos o a roles. Los usuarios las ven en la barra de notificaciones de la aplicación.

Campo	Tipo	Descripción
_id	ObjectId	Identificador único
title	String	Título breve de la notificación
body	String	Cuerpo del mensaje con el detalle

Campo	Tipo	Descripción
type	String (enum)	Tipo: ALERT, SYNC_COMPLETE, SYSTEM, INFO
read	Boolean	Indica si el usuario ya leyó la notificación
createdAt	DateTime	Fecha y hora de creación
regionId	String	Región relacionada (puede ser nulo para notificaciones globales)
userId	Long	ID del usuario destinatario (referencia a User.id en PostgreSQL)
alertId	String	Referencia a la alerta que originó la notificación (puede ser nulo)

Índices: { userId: 1, read: 1, createdAt: -1 } (compuesto), { regionId: 1, createdAt: -1 } (compuesto).

Colección: comunaRiskScores

Propósito: Almacena el score de riesgo WLC calculado por comuna. Cada documento representa el estado de riesgo más reciente de una comuna.

Campo	Tipo	Descripción
_id	ObjectId	Identificador interno
gadmGid	String	Identificador de la unidad comunal según GADM (ej. CHL.9.1_1)
nombreComuna	String	Nombre legible de la comuna
regionId	String	Referencia a la región a la que pertenece la comuna

Campo	Tipo	Descripción
scoreComposite	Double	Score WLC compuesto (0.0 – 1.0)
alertLevel	String (enum)	Nivel de alerta derivado del score: NORMAL, PREVENTIVO, ALTO, CRÍTICO
mode	String (enum)	Modo de cálculo: STANDARD o ENHANCED
components	Map	Mapa de componentes por variable (ver estructura abajo)
computedAt	DateTime	Timestamp del último cálculo del score
copernicusSyncedAt	DateTime	Timestamp del último sync con Copernicus/OpenEO

Estructura del campo components:

```
{
  "FWI":    { "score": 0.72, "weight": 0.30, "rawValue": 18.5 },
  "NDMI":   { "score": 0.45, "weight": 0.25, "rawValue": -0.12 },
  "NDVI":   { "score": 0.38, "weight": 0.20, "rawValue": 0.41 },
  "FIRMS":  { "score": 0.60, "weight": 0.15, "focosCount": 3 },
  "REPORTS": { "score": 0.20, "weight": 0.10, "count": 1 }
}
```

El scoreComposite se calcula como $\sum(\text{component.score} \times \text{component.weight})$.

Índices: { regionId: 1, alertLevel: 1 } (compuesto), { gadmGid: 1 } (único).

Colección: regionalRiskScores

Propósito: Almacena el score de riesgo consolidado a nivel regional, calculado como agregación de los ComunaRiskScore de todas las comunas de la región.

Campo	Tipo	Descripción
_id	ObjectId	Identificador interno
regionId	String	Referencia a la región (Region._id)
scoreComposite	Double	Score regional consolidado (0.0 – 1.0)
alertLevel	String (enum)	Nivel de alerta de la región
comunasCount	Integer	Total de comunas incluidas en el cálculo
comunasEnAlto	Integer	Cantidad de comunas con nivel ALTO
comunasEnCritico	Integer	Cantidad de comunas con nivel CRÍTICO
generatedAt	DateTime	Timestamp de la última generación del score regional

Índices: { regionId: 1, generatedAt: -1 } (compuesto), { alertLevel: 1 } (simple).

Resumen de colecciones MongoDB al cierre de EP3:

Colección	Estado	Propósito
regions	Existente (actualizada)	Regiones monitoreadas con monitoringEnabled
alertRules	Nueva (EP3)	Reglas de alerta configurables por región

Colección	Estado	Propósito
alerts	Nueva (EP3)	Alertas generadas automáticamente
notifications	Nueva (EP3)	Notificaciones para usuarios
comunaRiskScores	Nueva (EP3)	Score WLC por comuna con componentes
regionalRiskScores	Nueva (EP3)	Score regional consolidado
communityPosts	Existente	Publicaciones del tablero comunitario
communityContacts	Existente (corregida EP3)	Agenda de contactos con comunald
communityResources	Existente	Biblioteca de recursos (solo PDF desde EP3)
citizenReports	Existente	Reportes ciudadanos de incidentes
forestLossRecords	Existente	Registros históricos de pérdida forestal
firmsDetections	Existente	Focos activos sincronizados desde NASA FIRMS
weatherReadings	Existente	Lecturas meteorológicas (FWI, viento, humedad)

7.3 Guía de instalación y despliegue

La guía completa con Docker Compose, configuración Nginx y resolución de problemas frecuentes se encuentra en [Documentacion/2026-06-20_guia_instalacion_despliegue.md](#).

Resumen ejecutivo de instalación:

Producción actual:

- Frontend: Vercel (CD automático desde rama main)
- Backend: Railway (JAR Spring Boot)
- Servicio OpenEO: Railway (Python/FastAPI)
- Base de datos documental: MongoDB Atlas (M0, 512 MB gratuito)
- Base de datos relacional: PostgreSQL Supabase (plan gratuito, 500 MB)

Requisitos para instalación local:

Componente	Versión mínima
Java	21 (LTS)
Maven	3.9+
Node.js	20 (LTS)
npm	10+
MongoDB	7.0+
PostgreSQL	15+

Pasos de instalación local (resumen):

1. Clonar el repositorio y configurar application.properties con credenciales locales.
2. Crear base de datos PostgreSQL simfat y usuario simfat_user (Flyway aplica el esquema automáticamente al iniciar).
3. Iniciar MongoDB en modo standalone local (la base de datos simfat se crea automáticamente).
4. Levantar el backend: `cd Producto/backend/simfat-api && ./mvnw spring-boot:run`
5. Instalar dependencias del frontend y levantar: `cd Producto/frontend/simfat-web && npm install && npm run dev`

Variables de entorno críticas:

Variable	Descripción
SPRING_DATASOURCE_URL	URL JDBC de PostgreSQL
SPRING_DATA_MONGODB_URI	URI de conexión a MongoDB
JWT_SECRET	Clave secreta para firma de tokens (mínimo 256 bits)
FIRMS_API_KEY	Clave NASA Earthdata para FIRMS
OPENWEATHERMAP_API_KEY	Clave OpenWeatherMap para FWI
VITE_API_URL	URL del backend desde el frontend

Verificación post-instalación:

Verificación	URL
Health check del backend	GET http://localhost:8080/actuator/health → {"status":"UP"}
Swagger UI	http://localhost:8080/swagger-ui.html
Frontend	http://localhost:5173

Instalación con Docker Compose: disponible en el repositorio con el archivo docker-compose.yml en la raíz del proyecto. Levanta el stack completo (PostgreSQL, MongoDB, backend, frontend) con un único comando docker compose up --build -d.

7.4 Control de versiones

- **Repositorio:** GitHub (organización SIMFAT-2026)
- **Rama principal:** main
- **Estrategia de ramas:** main (rama estable de producción), ramas de trabajo por dominio (backend/*, frontend/*, openeo-service/*)

- **Convención de commits:** Conventional Commits
— feat, fix, docs, chore, refactor

Commits EP3 relevantes:

Hash	Mensaje	Descripción
65b9484	fix(community): restrict resource library uploads to PDF only	Restricción de subida de archivos a solo PDF en la biblioteca de recursos
7842ebd	feat(alerts): align AlertRule thresholds with WLC risk score variables	Alineación de umbrales con las variables del score WLC y rangos de ayuda
d2a7f23	docs(qa): document CU09/CU15 production QA run	Documentación del run de QA en producción para CU09 y CU15
57314e9	refactor(territory): extract colorblind-safe risk color scales	Extracción de escalas de color colorblind-safe como módulo reutilizable
7842ebd	feat(community): add comuna to contacts and revamp contacts agenda UI	Corrección del payload de contactos con comunald y renovación de la UI de agenda

8. Estado MVP en Producción

Componente	Plataforma	Estado
Frontend web	Vercel	Operativo
Backend API	Railway	Operativo
Servicio OpenEO	Railway	Operativo (parcial)

Componente	Plataforma	Estado
MongoDB Atlas	Cloud (M0)	Operativo
PostgreSQL	Supabase	Operativo
Swagger UI	Railway	Operativo

URLs de producción:

Servicio	URL
Frontend	https://simfat-web-stg.vercel.app/
Backend API	https://simfat-backend-production.up.railway.app
Servicio OpenEO	https://openeo-service-production-production.up.railway.app
Swagger UI	https://simfat-backend-production.up.railway.app/swagger-ui/index.html

El servicio OpenEO se marca como "Operativo (parcial)" porque el sync programado diario está activo, pero el sync on-demand desde el panel comunal puede fallar ocasionalmente por restricciones de memoria en el plan gratuito de Railway. El sync por demanda de datos ENHANCED fue estabilizado durante EP3 corrigiendo el problema de timeout en la red interna de Railway.

9. Costos y Sostenibilidad

La infraestructura actual de SIMFAT opera en su totalidad sobre planes gratuitos o de costo mínimo, lo que es coherente con su propósito académico y de prototipo.

Servicio	Plan	Costo actual
Vercel (frontend)	Hobby (gratuito)	\$0 USD/mes

Servicio	Plan	Costo actual
Railway (backend + openeo)	Starter (créditos agotados → billing)	~\$5-10 USD/mes
MongoDB Atlas (M0)	Free tier — 512 MB	\$0 USD/mes
Supabase (PostgreSQL)	Free tier — 500 MB	\$0 USD/mes
Total estimado		\$0 – \$10 USD/mes

El único servicio con costo real es Railway, donde se alojan el backend Spring Boot y el servicio OpenEO. Los créditos de inicio de Railway se agotaron durante el período de EP3, lo que significa que el proyecto incurre en un costo mensual aproximado de \$5-10 USD dependiendo del uso de recursos y el tiempo de actividad del servicio OpenEO.

Recomendación para post-entrega: Mantener la infraestructura actual durante el período de presentación y defensa. Evaluar migración a Oracle Cloud Free Tier (que ofrece máquinas virtuales con 1 OCPU y 1 GB RAM sin caducidad de créditos) para reducir el costo a \$0 en producción sostenida.

Escalabilidad: El diseño actual soporta el volumen de datos del proyecto académico. Para una eventual puesta en producción real con AIFBN se requerirá:

- MongoDB Atlas M2 o superior (\$9 USD/mes, 2 GB) para el histórico de scores y detecciones FIRMS.
- Railway Pro o equivalente (\$20 USD/mes) para garantizar SLA de uptime del backend y el servicio OpenEO.
- Supabase Pro (\$25 USD/mes) si el volumen de usuarios supera los 50,000 registros.

10. Conclusiones

Logro de objetivos: SIMFAT logró implementar los tres objetivos específicos planteados en EP1. Primero, la integración de datos: el sistema consume en tiempo casi real los datos de NASA FIRMS, Open-Meteo (FWI), Copernicus/OpenEO (NDVI/NDMI) y los reportes ciudadanos internos, normalizándolos en un modelo de datos unificado en MongoDB. Segundo, el

dashboard analítico: el panel regional embebido en la página de territorio proporciona a los coordinadores una vista consolidada de KPIs de riesgo, vegetación y clima actualizados cada 12 horas. Tercero, las alertas tempranas: el sistema de reglas configurables evalúa automáticamente los indicadores en cada ciclo de sync y genera alertas con notificaciones cuando los umbrales definidos son superados.

Evolución EP1 a EP3: El proyecto pasó del prototipo conceptual documentado en EP1 a un MVP online con datos reales satelitales, meteorológicos y comunitarios. En EP2 se construyeron los cimientos de seguridad (RBAC/JWT). En EP3 se completaron los módulos de cara al usuario: mapa territorial con análisis WLC, panel analítico regional, módulo de alertas y módulo comunitario completo. La arquitectura dual PostgreSQL/MongoDB establecida en EP2 se validó en EP3 como decisión correcta: permitió evolucionar el modelo de datos de negocio (nuevas colecciones de scores, alertas y notificaciones) sin afectar el sistema de identidad y acceso.

Calidad del producto: El plan formal de pruebas de EP3 registró un 88.2% de casos aprobados (30 de 34). Los 2 casos OBSERVADO no bloquean funcionalidad y están implementados, a la espera de validación en producción con usuarios reales. Los 2 casos PENDIENTE corresponden a funcionalidades de backend que dependen de condiciones de entorno no reproducibles en QA manual controlado (WebSocket para chat y cron job con datos reales para notificaciones). El ciclo de 9 mejoras derivadas del QA demuestra un proceso de calidad continuo trazable desde la detección del problema hasta la corrección implementada.

Valor para el cliente AIFBN: La plataforma permite a los coordinadores forestales de AIFBN monitorear el riesgo de incendio forestal por región y comuna con datos actualizados automáticamente dos veces al día. El score WLC proporciona una métrica compuesta que integra múltiples fuentes en un indicador de 0 a 100 con niveles de interpretación claros (NORMAL/PREVENTIVO/ALTO/CRÍTICO). La confirmación satelital on-demand con Copernicus permite elevar la precisión del análisis cuando las condiciones lo requieren, en un flujo de trabajo integrado directamente en el mapa territorial. Los informes PDF por región y comuna facilitan la comunicación de los coordinadores con equipos de terreno.

Deuda técnica controlada: Las funcionalidades pendientes son dos y están claramente acotadas: el chat en tiempo real con WebSocket (implementado en frontend y backend, pendiente estabilización en Railway) y las notificaciones automáticas por AlertRule (infraestructura implementada, pendiente activación del cron job de evaluación con datos reales en producción). Ambas son mejoras incrementales sobre una base funcional, no bloqueantes para el uso del sistema.

11. Lecciones Aprendidas

1. **La integración de fuentes de datos externas requiere normalización defensiva.** NASA FIRMS, Open-Meteo y Copernicus tienen formatos de respuesta completamente distintos, frecuencias de actualización diferentes y pueden estar caídas o demoradas sin aviso. El patrón de diseño correcto es siempre normalizar a un modelo interno antes de persistir, y nunca asumir que la API externa estará disponible. La ausencia de esta disciplina causó bugs silenciosos de datos que tardaron varios sprints en ser identificados.
2. **RBAC data-driven (permisos en base de datos, no en código) permite evolucionar sin redeploy del backend.** La decisión de almacenar roles y permisos en PostgreSQL en lugar de hardcodearlos en anotaciones Spring se validó en EP3 al agregar el acceso al chat comunitario por región sin necesidad de tocar el código del backend. El costo de implementación inicial fue mayor, pero el beneficio de mantenibilidad a largo plazo es claro.
3. **El scoring WLC debe documentarse matemáticamente desde el inicio del proyecto.** Los pesos de las variables del score de riesgo cambiaron varias veces durante el desarrollo (de EP2 a EP3), y cada cambio afectó el frontend (visualización del desglose), el backend (cálculo) y los informes PDF (explicación de la contribución de cada variable). Sin documentación formal de la fórmula y sus pesos en cada versión, hubiera sido imposible auditar los cambios.
4. **Los errores silenciosos en el frontend son los más difíciles de diagnosticar en producción.** El bug de la agenda de contactos (error HTTP 400 al guardar) pasó desapercibido durante semanas porque el catch en el formulario no mostraba el error al usuario: los contactos "parecían guardarse" pero desaparecían al recargar la página. La lección es: nunca tener catch vacíos ni catches que solo hagan console.error. El error debe llegar al usuario de forma clara.
5. **El filtro client-side de FIRMS es una solución aceptable a corto plazo, no una definitiva.** Filtrar los focos FIRMS por bounding box en el frontend resolvió el problema inmediato del conteo incorrecto, pero la solución correcta es que el backend filtre por regionId al persistir los datos. La deuda técnica quedó documentada para la siguiente iteración.
6. **Separar el ambiente de staging del de producción fue crítico para la estabilidad de las demos.** Railway experimentó eventos OOM (Out of

Memory) durante el sprint de EP3. Sin un ambiente de staging separado, cada evento habría afectado directamente la demo ante el cliente. La lección es: para proyectos académicos con demos frecuentes, el ambiente de presentación debe protegerse de cambios experimentales.

7. **La documentación técnica generada como artefacto de diseño (SDD, contratos de arquitectura) aceleró la implementación más que los comentarios en el código.** Los documentos SDD del módulo territorial y del sistema RBAC permitieron al equipo retomar el contexto de cada sprint sin necesidad de releer el código. La disciplina de documentar antes de implementar, aunque requiere esfuerzo inicial, amortiza ese costo en cada sprint posterior.
8. **Copernicus/OpenEO tarda 2-3 minutos por proceso — comunicarlo claramente en la UI previene frustración.** Durante las pruebas tempranas de la confirmación satelital, la ausencia de un indicador de progreso claro llevó a los usuarios a hacer clic varias veces en el botón de sync creyendo que no respondía. El feedback al usuario durante operaciones asíncronas largas no es un "nice to have", es parte de la funcionalidad.
9. **Los informes PDF generados con window.print() nativo eliminaron dependencias costosas.** La evaluación inicial del stack de generación de PDF (jsPDF, Puppeteer) mostraba complejidad de integración y tamaño de bundle significativo. La decisión de usar la API nativa del navegador simplificó el deployment, redujo el bundle size y produjo informes de mejor calidad visual porque reutiliza el CSS existente de la aplicación.
10. **Mantener trazabilidad bidireccional entre pruebas y mejoras convierte el QA en una herramienta de gestión, no solo de validación.** La tabla de mejoras con la columna de caso de prueba relacionado permitió al equipo comunicar al cliente no solo qué se corrigió, sino por qué se corrigió y cuándo fue detectado. Esta trazabilidad es la que transforma el proceso de QA de una actividad de verificación a un proceso de mejora continua documentada.

12. Anexos

Anexo A — Evidencias QA (capturas de pantalla)

1. Creación de una nueva publicación en el mural comunitario.

Nueva publicación

Crea una nota breve para pegarla en el mural comunitario.

Título

Prioridad Region

CRITICA

Ñuble

Mensaje

Esto es una prueba

Seleccionar archivo

DOCUMENTEN...L FASE 1.pdf

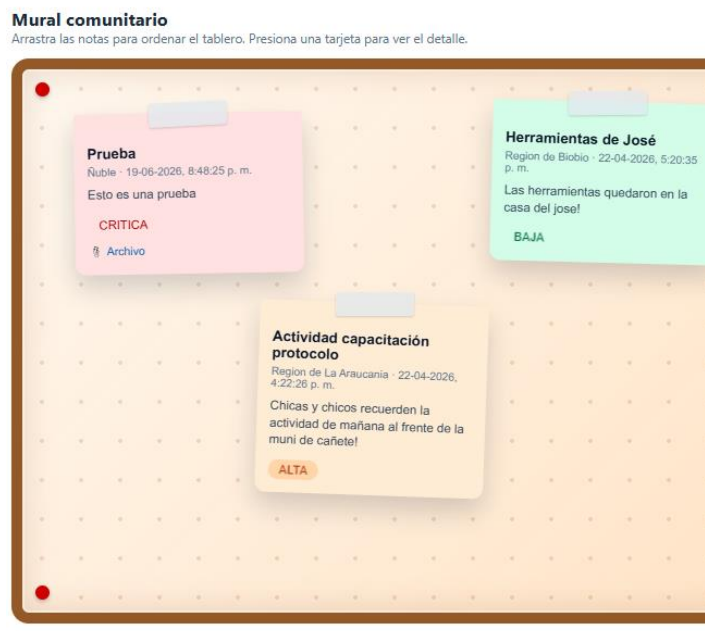
DOCUMENTOS FINAL FAS...

4047 KB

Quitar

Pegar en mural

- Validación del proceso de creación de publicaciones en la Biblioteca de Recursos.



- Verificación del comportamiento del sistema al publicar recursos asociados a regiones con comunas no configuradas.

Biblioteca de recursos

Sube documentos PDF o DOCX y asocalos a una region y comuna.

Titulo

Prueba 2

Categoria

PROTOCOLO

Region

Region de Antofagasta

Comuna

Seleccione comuna

No hay comunas configuradas para esta region.

Descripcion

Seleccionar archivo Sin archivos seleccionados

Formatos permitidos: PDF y DOCX.

Subir recurso

MATERIAL

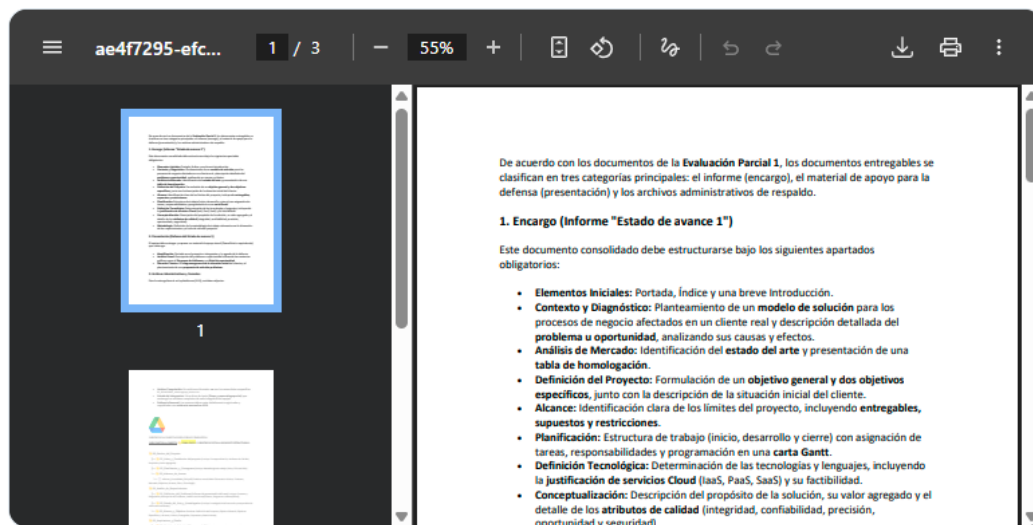
Prueba 1

Abrir

Eliminar

Biobio - Coronel

Esto es una prueba para la biblioteca de recursos con archivo pdf adjunto



ae4f7295-efc... 1 / 3 55%

De acuerdo con los documentos de la **Evaluación Parcial 1**, los documentos entregables se clasifican en tres categorías principales: el informe (encargo), el material de apoyo para la defensa (presentación) y los archivos administrativos de respaldo.

1. Encargo (Informe "Estado de avance 1")

Este documento consolidado debe estructurarse bajo los siguientes apartados obligatorios:

- **Elementos Iniciales:** Portada, índice y una breve Introducción.
- **Contexto y Diagnóstico:** Planteamiento de un **modelo de solución** para los procesos de negocio afectados en un cliente real y descripción detallada del **problema u oportunidad**, analizando sus causas y efectos.
- **Análisis de Mercado:** Identificación del **estado del arte** y presentación de una **tabla de homologación**.
- **Definición del Proyecto:** Formulación de un **objetivo general** y **dos objetivos específicos**, junto con la descripción de la situación inicial del cliente.
- **Alcance:** Identificación clara de los límites del proyecto, incluyendo **entregables, supuestos y restricciones**.
- **Planificación:** Estructura de trabajo (inicio, desarrollo y cierre) con asignación de tareas, responsabilidades y programación en una **carta Gantt**.
- **Definición Tecnológica:** Determinación de las tecnologías y lenguajes, incluyendo la **justificación de servicios Cloud** (IaaS, PaaS, SaaS) y su factibilidad.
- **Conceptualización:** Descripción del propósito de la solución, su valor agregado y el detalle de los **atributos de calidad** (integridad, confiabilidad, precisión, oportunidad y seguridad).

4. Evaluación de la creación de publicaciones sin adjuntar documentación de respaldo.

Biblioteca de recursos

Sube documentos PDF o DOCX y asocalos a una region y comuna.

Título

Categoría

PROTOCOLO

▼

Region

Region de Nuble

▼

Comuna

Cobquecura

▼

Descripción

Esto es una prueba sin documento adjunto

Seleccionar archivo

Sin archivos seleccionados

Formatos permitidos

!

Selecciona un archivo.

Subir recurso

5. Validación del proceso de registro y gestión de contactos y protocolos comunitarios.

Contactos y protocolos

Registra responsables con teléfono, WhatsApp, correo y protocolo de escalamiento.

Nombre

Organización

Teléfono

Correo

Region

Region de Biobio

▼

Comuna

Cabrero

▼

Protocolo

Esto es una prueba

Guardar contacto

Agenda de contactos

Responsables y canales de escalamiento por territorio.

Buscar

Region

Todas

Comuna

Seleccione region

Forestin
Forestal
Biobio - Sin comuna

+56966666666


Eliminar

Esto es una prueba

- Verificación de la configuración territorial para la Zona Metropolitana en ausencia de comunas asociadas.

Contactos y protocolos

Registra responsables con teléfono, WhatsApp, correo y protocolo de escalamiento.

Nombre

Organizacion

Telefono

Correo

Region

Region de Metropolitana de Santiago

Comuna

Seleccione comuna

Protocolo

Guardar contacto

- Evaluación de las restricciones de acceso al Chat Comunitario para usuarios sin identidad verificada.

Chat comunitarioOcultar

Sala

Coordinacion territorial para prevencion de incendios.

Estado

Conectado

Debes tener identidad verificada para usar el chat comunitario

Sin mensajes

Inicia la coordinacion en esta sala.

Escribe una actualizacion operativa...

Enviar

8. Validación del proceso de generación de reportes ciudadanos sin asignación de comuna.

Reportes ciudadanos

Ingreso geolocalizado con evidencia y seguimiento operativo

Origen de datos: backend de reportes.

Ubicacion capturada correctamente.

Region	Estado	Categoria	Recargar
Region de Los Rios	VALIDADO	QUEMA	

Nuevo reporte

Region	Comuna	Categoria	Subcategoria
Ñuble	Seleccione comuna	COMBUSTIBLE_VEGETAL	Pastizales secos sin manejo

Descripcion

Esto es una prueba sin comuna

! Seleccione un elemento de la lista

Fotos (max 4, 5 MB c/u)

[Elegir archivos](#) Sin archivos seleccionados

[Usar mi ubicacion](#)

[Enviar reporte](#)

Seguimiento de reportes

Sin reportes

- Verificación de controles de validación ante la ausencia de subcategorías en el registro de información.

Region	Estado	Categoria	Recargar
Region de Los Rios	VALIDADO	QUEMA	

Nuevo reporte

Region	Comuna	Categoria	Subcategoria	Latitu
Ñuble	San Nicolás	IGNICION_POTENCIAL	Seleccione subcategoria	

Descripcion

! Seleccione un elemento de la lista

Fotos (max 4, 5 MB c/u)

[Elegir archivos](#) Sin archivos seleccionados

[Usar mi ubicacion](#)

[Enviar reporte](#)

- Evaluación del comportamiento del sistema cuando no se proporciona información de geolocalización (latitud).

Reportes ciudadanos

ngreso geolocalizado con evidencia y seguimiento operativo

Origen de datos: backend de reportes.

Reporte ciudadano enviado correctamente.

Region

Estado

Categoria

Recargar

Region de Los Rios

VALIDADO

QUEMA

Nuevo reporte

Region

Comuna

Categoria

Latitud

Lo

Ñuble

San Nicolás

OTRO

Descripción

Prueba con foto

Completar este campo

Fotos (max 4, 5 MB c/u)

Elegir archivos

WhatsApp Image 2025-10-18 at 12.56.59 AM (1).jpeg

WhatsApp Image 2025-10-18 at 12.56.59 AM (1).jpeg

Usar mi ubicación

Enviar reporte

11. Validación del proceso de creación y gestión de alertas comunitarias.

Alertas

Eventos de calor, priorización territorial y cruce con reportes ciudadanos

Filtrar por region

Nivel de riesgo

Desde

Hasta

Actualizar cruce

Limpiar filtros

Ver en territorio

Todas

Todos

dd-mm-aaaa

dd-mm-aaaa

Alertas filtradas

Criticas / Altas

Reportes (total / validados)

Regiones con alerta

0

0 / 0

0 / 0

0

Region

Fecha evento

Nivel riesgo

Latitud

Longitud

Fuente

Ñuble

20-06-2026 22:16

CRITICO

Descripción

Crear

timeout of 15000ms exceeded

Retry

Las capturas deben incluir: página de login (CU01), mapa territorial con choropleth activo (CU02), panel comunal con score WLC (CU04), formulario de reglas de alerta (CU05), tablero comunitario (CU07), agenda de contactos con comunald (CU09), panel de accesos RBAC (CU14), y administración de regiones con toggle (CU15).

Anexo B — Capturas del sistema en producción

Pendiente — capturas de los módulos principales: mapa territorial, panel comunal, alertas, dashboard y RBAC.

Anexo C — Código fuente

El código fuente completo está disponible en el repositorio GitHub de la organización SIMFAT-2026. Rama principal: main.

Estructura del repositorio:

simfat-main/

```
├── Producto/
|   ├── backend/
|   |   ├── simfat-api/      # API Spring Boot (Java 21)
|   |   └── frontend/
|   └── simfat-web/         # Aplicación React + Vite
├── Documentacion/         # Documentación académica y técnica
└── docker-compose.yml
```

Anexo D — Plan de pruebas completo (referencia)

Ver: Documentacion/Evidencias/2026-06-20_plan_pruebas_EP3.md

Contiene la tabla original de los 34 casos de prueba con los resultados obtenidos en el ambiente de producción, los casos OBSERVADO detallados y los casos PENDIENTE con descripción de la condición bloqueante.

Anexo E — Tabla de mejoras completa (referencia)

Ver: Documentacion/Evidencias/2026-06-20_mejoras_EP3.md

Contiene la tabla de 9 mejoras con la descripción de cada una, el módulo afectado, el impacto en el producto y la tabla de trazabilidad con el caso de prueba relacionado.

Anexo F — Guía de instalación y despliegue

Ver: Documentacion/2026-06-20_guia_instalacion_despliegue.md

Incluye requisitos completos, configuración de variables de entorno, instrucciones para instalación sin Docker, instrucciones para instalación con Docker Compose, instrucciones de despliegue en producción (Vercel, Railway, Nginx), verificación post-instalación y tabla de resolución de problemas frecuentes.

Anexo G — Diagrama de clases (referencia)

Ver: Documentacion/UML/2026-06-20_diagrama_clases_ep3.md

Contiene el diagrama Mermaid completo con notas de diseño sobre la separación PostgreSQL/MongoDB, la estructura del campo components de ComunaRiskScore y los criterios de determinación del nivel de alerta.

Anexo H — Modelo lógico MER EP3 (referencia)

Ver: Documentacion/MER/2026-06-20_nuevas_colecciones_EP3.md

Documenta las 5 colecciones nuevas de MongoDB de EP3 con descripción completa de campos, índices y relaciones entre colecciones.